UNIT IV

Measures of Query cost

Cost of query is the time taken by the query to hit the database and return the result. It involves query processing time i.e.; time taken to parse and translate the query, optimize it, evaluate, execute and return the result to the user is called cost of the query. Though it is in fraction of seconds, it includes multiple sub tasks and time taken by each of them. Executing the optimized query involves hitting the primary and secondary memory based on the file organization method. Depending on file organization and the indexes used, time taken to retrieve the data may vary.

Majority of time is spent by the query in accessing the data from the memory. It too has several factors determining the cost of access time – disk I/O time, CPU time, network access time etc. Disk access time is the time taken by the processor to search and find the record in the secondary memory and return the result. This takes the majority of time while processing a query. Other times can be ignored compared to disk I/O time.

While calculating the disk I/O time, usually only two factors are considered – seek time and transfer time. The **seek time** is the time taken the processor to find a single record in the disk memory and is represented by tS. For example, in order to find the student ID of a student 'John', the processor will fetch in the memory based on the index and the file organization method. The time taken by the processor to hit the disk block and search for his ID is called the seek time. The time taken by the disk to return fetched result back to the processor / user is called transfer time and is represented by tT. Suppose a query need to seek S times to fetch a record and there is B blocks needs to be returned to the user. Then the disk I/O cost is calculated as below

(S* tS)+ (B* tT)

That is, it is the sum of the total time taken for seek S times and the total time taken to transfer B blocks. Here other costs like CPU cost, RAM cost etc are ignored as they are comparatively small. Disk I/O alone is considered as cost of a query. But we have to calculate the worst case cost – the maximum time taken by the query when there is a worst case like buffer is full or no buffers etc. because the memory space / buffers depend on the number of queries executing in parallel. All queries would be using the buffers and determining the number of buffers / blocks available for our query is unpredictable. The processor might have to wait till it gets all the memory blocks.

Query Processing in DBMS

Query Processing is the activity performed in extracting data from the database. In query processing, it takes various steps for fetching the data from the database. The steps involved are:

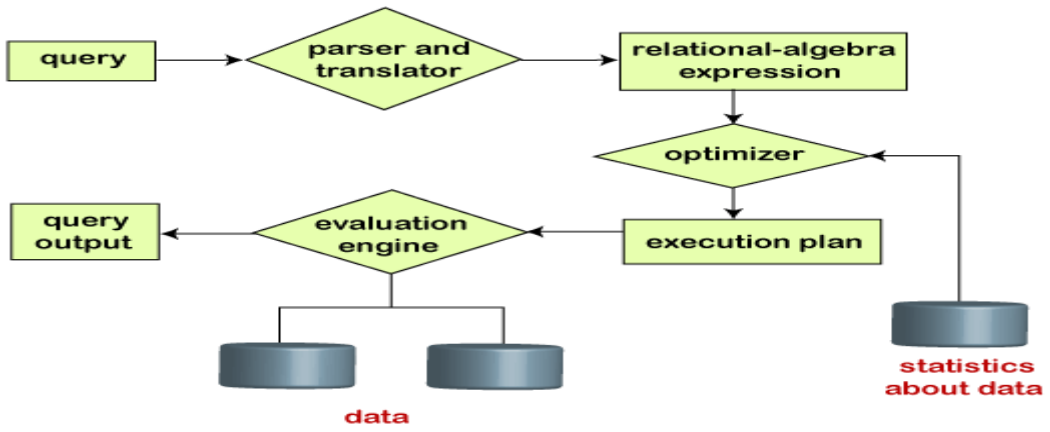
1. Parsing and translation
2. Optimization
3. Evaluation

The query processing works in the following way:

Parsing and Translation:

As query processing includes certain activities for data retrieval. Initially, the given user queries get translated in high-level database languages such as SQL. It gets translated into expressions that can be further used at the physical level of the file system. After this, the actual evaluation of the queries and a variety of query -optimizing transformations and takes place. Thus before processing a query, a computer system needs to translate the query into a human-readable and understandable language. Consequently, SQL or Structured Query Language is the best suitable choice for humans. But, it is not perfectly suitable for the internal representation of the query to the system. Relational algebra is well suited for the internal representation of a query. The translation process in query processing is similar to the parser of a query. When a user executes any query, for generating the internal form of the query, the parser in the system checks the syntax of the query, verifies the name of the relation in the database, the tuple, and finally the required attribute value. The parser creates a tree of the query, known as 'parse-tree.' Further, translate it into the form of relational algebra. With this, it evenly replaces all the use of the views when used in the query.

Thus, we can understand the working of a query processing in the below-described diagram:



Steps in query processing

Suppose a user executes a query. As we have learned that there are various methods of extracting the data from the database. In SQL, a user wants to fetch the records of the employees whose salary is greater than or equal to 10000. For doing this, the following query is undertaken:

select emp_name from Employee where salary>10000;

Thus, to make the system understand the user query, it needs to be translated in the form of relational algebra. We can bring this query in the relational algebra form as:

- $\sigma_{salary>10000}$ (π_{salary} (Employee))
- π_{salary} ($\sigma_{salary>10000}$ (Employee))

After translating the given query, we can execute each relational algebra operation by using different algorithms. So, in this way, a query processing begins its working.

Evaluation:

For this, with addition to the relational algebra translation, it is required to annotate the translated relational algebra expression with the instructions used for specifying and evaluating each operation. Thus, after translating the user query, the system executes a query evaluation plan.

Query Evaluation Plan:

- In order to fully evaluate a query, the system needs to construct a query evaluation plan.
- The annotations in the evaluation plan may refer to the algorithms to be used for the particular index or the specific operations.

- Such relational algebra with annotations is referred to as **Evaluation Primitives**. The evaluation primitives carry the instructions needed for the evaluation of the operation.
- Thus, a query evaluation plan defines a sequence of primitive operations used for evaluating a query. The query evaluation plan is also referred to as **the query execution plan**.
- A **query execution engine** is responsible for generating the output of the given query. It takes the query execution plan, executes it, and finally makes the output for the user query.

Optimization:

- The cost of the query evaluation can vary for different types of queries. Although the system is responsible for constructing the evaluation plan, the user does need not to write their query efficiently.
- Usually, a database system generates an efficient query evaluation plan, which minimizes its cost. This type of task performed by the database system and is known as Query Optimization.
- For optimizing a query, the query optimizer should have an estimated cost analysis of each operation. It is because the overall operation cost depends on the memory allocations to several operations, execution costs, and so on.

Finally, after selecting an evaluation plan, the system evaluates the query and produces the output of the query.

There are two methods of evaluating the query.

Materialization:

In this method, queries are broken into individual queries and then the results of which are used to get the final result. To be more specific, suppose there is a requirement to find the students who are studying in class 'DESIGN_01'.

```
SELECT * FROM STUDENT s, CLASS c
```

```
WHERE s.CLASS_ID = c.CLASS_ID AND c.CLASS_NAME = 'DESIGN_01';
```

Here we can observe two queries: one is to select the CLASS_ID of 'DESIGN_01' and another is to select the student details of the CLASS_ID retrieved in the first query.
The DBMS also does the same. It breaks the query into two as mentioned above. Once it is broken, it evaluates the first query and stores it in the temporary table in the memory. This temporary table data will be then used to evaluate the second query.

This is the example of two level queries in materialization method. We can have any number of levels and so many numbers of temporary tables.

Although this method looks simple, the cost of this type of evaluation is always more. It takes the time to evaluate and write into temporary table, then retrieve from this temporary table and query to get the next level of result and so on. Hence cost of evaluation in this method is:
Cost = cost of individual SELECT + cost of write into temporary table

Pipelining:

In this method, DBMS do not store the records into temporary tables. Instead, it queries each query and result of which will be passed to next query to process and so on. It will process the query one after the other and each will use the result of previous query for its processing.

In the example above, CLASS_ID of DESIGN_01 is passed to the STUDENT table to get the student details.

In this method no extra cost of writing into temporary tables. It has only cost of evaluation of individual queries; hence it has better performance than materialization.

There are two types of pipelining:

Demand Driven or Lazy evaluation

In this method, the result of lower level queries are not passed to the higher level automatically. It will be passed to higher level only when it is requested by the higher level. In this method, it retains the result value and state with it and it will be transferred to the next level only when it is requested.

In our example above, CLASS_ID for DESIGN_01 will be retrieved, but it will not be passed to STUDENT query only when it is requested. Once it gets the request, it is passed to student query and that query will be processed.

Producer Driven or Eager Pipelining

In this method, the lower level queries eagerly pass the results to higher level queries. It does not wait for the higher level queries to request for the results. In this method, lower level query creates a buffer to store the results and the higher level queries pulls the results for its use. If the buffer is full, then the lower level query waits for the higher level query to empty it. Hence it is also called as PULL and PUSH pipelining.

There are still more methods of pipelining like Linear and non-linear methods of pipelining, left deep tree, right deep tree etc.

Query Optimization in DBMS

We have seen so far how a query can be processed based on indexes and joins, and how they can be transformed into relational expressions. The query optimizer uses these two techniques to determine which process or expression to consider for evaluating the query.

There are two methods of query optimization.

1. Cost based Optimization (Physical)

This is based on the cost of the query. The query can use different paths based on indexes, constraints, sorting methods etc. This method mainly uses the statistics like record size, number of records, number of records per block, number of blocks, table size, whether whole table fits in a block, organization of tables, uniqueness of column values, size of columns etc.
Suppose, we have series of table joined in a query.

T1 ∞ T2 ∞ T3 ∞ T4∞ T5 ∞ T6
For above query we can have any order of evaluation. We can start taking any two tables in any order and start evaluating the query. Ideally, we can have join combinations in (2(n-1))! / (n-1)! ways. For example, suppose we have 5 tables involved in join, then we can have 8! / 4! = 1680 combinations. But when query optimizer runs, it does not evaluate in all these ways always. It uses Dynamic Programming where it generates the costs for join orders of any combination of tables. It is calculated and generated only once. This least cost for all the table combination is then stored in the database and is used for future use. i.e.; say we have a set of tables, T = { T1 ,

T2 , T3 .. Tn}, then it generates least cost combination for all the tables and stores it.

- Dynamic Programming

As we learnt above, the least cost for the joins of any combination of table is generated here. These values are stored in the database and when those tables are used in the query, this combination is selected for evaluating the query.
While generating the cost, it follows below steps :
Suppose we have set of tables, T = {T1 , T2 , T3 .. Tn}, in a DB. It picks the first table, and computes cost for joining with rest of the tables in set T. It calculates cost for each of the tables and then chooses the best cost. It continues doing the same with rest of the tables in set T. It will generate 2n – 1 cases and it selects the lowest cost and stores it. When a query uses those tables, it checks for the costs here and that combination is used to evaluate the query. This is called dynamic programming.
In this method, time required to find optimized query is in the order of 3n, where n is the number of tables. Suppose we have 5 tables, then time required in 35 = 243, which is lesser than finding all the combination of tables and then deciding the best combination (1680). Also, the space required for computing and storing the cost is also less and is in the order of 2n. In above example, it is 25 = 32.

- Left Deep Trees

This is another method of determining the cost of the joins. Here, the tables and joins are represented in the form of trees. The joins always form the root of the tree and table is kept at the right side of the root. LHS of the root always point to the next join. Hence it gets deeper and deeper on LHS. Hence it is called as left deep tree.

Here instead of calculating the best join cost for set of tables, best join cost for joining with each table is calculated. In this method, time required to find optimized query is in the order of n2n, where n is the number of tables. Suppose we have 5 tables, then time required in 5*25 =160, which is lesser than dynamic programming. Also, the space required for computing storing the cost is also less and is in the order of 2n. In above example, it is 25 = 32, same as dynamic programming.

- Interesting Sort Orders

This method is an enhancement to dynamic programming. Here, while calculating the best join order costs, it also considers the sorted tables. It assumes, calculating the join orders on sorted tables would be efficient. i.e.; suppose we have unsorted tables T1 , T2 , T3 .. Tn and we have join on these tables.
(T1 ∞T2)∞ T3 ∞… ∞ Tn
This method uses hash join or merge join method to calculate the cost. Hash Join will simply join the tables. We get sorted output in merge join method, but it is costlier than hash join. Even though merge join is costlier at this stage, when it moves to join with third table, the join will have less effort to sort the tables. This is because first table is the sorted result of first two tables. Hence it will reduce the total cost of the query.
But the number of tables involved in the join would be relatively less and this cost/space difference will be hardly noticeable.

All these cost based optimizations are expensive and are suitable for large number of data. There is another method of optimization called heuristic optimization, which is better compared to cost based optimization.

2. Heuristic Optimization (Logical)

This method is also known as rule based optimization. This is based on the equivalence rule on relational expressions; hence the number of combination of queries get reduces here. Hence the cost of the query too reduces.

This method creates relational tree for the given query based on the equivalence rules. These equivalence rules by providing an alternative way of writing and evaluating the query, gives the better path to evaluate the query. This rule need not be true in all cases. It needs to be examined after applying those rules. The most important set of rules followed in this method is listed below:

- Perform all the selection operation as early as possible in the query. This should be first and foremost set of actions on the tables in the query. By performing the selection operation, we can reduce the number of records involved in the query, rather than using the whole tables throughout the query.

Suppose we have a query to retrieve the students with age 18 and studying in class DESIGN_01. We can get all the student details from STUDENT table, and class details from CLASS table. We can write this query in two different ways.

Here both the queries will return same result. But when we observe them closely we can see that first query will join the two tables first and then applies the filters. That means, it traverses whole table to join, hence the number of records involved is more. But he second query, applies the filters on each table first. This reduces the number of records on each table (in class table, the number of record reduces to one in this case!). Then it joins these intermediary tables. Hence the cost in this case is comparatively less.

Instead of writing query the optimizer creates relational algebra and tree for above case.

- Perform all the projection as early as possible in the query. This is similar to selection but will reduce the number of columns in the query.

Suppose for example, we have to select only student name, address and class name of students with age 18 from STUDENT and CLASS tables.

Here again, both the queries look alike, results alike. But when we compare the number of records and attributes involved at each stage, second query uses less records and hence more efficient.

- Next step is to perform most restrictive joins and selection operations. When we say most restrictive joins and selection means, select those set of tables and views which will result in comparatively less number of records. Any query will have better performance when tables with few records are joined. Hence throughout heuristic method of optimization,

the rules are formed to get less number of records at each stage, so that query performance is better. So is the case here too.

Suppose we have STUDENT, CLASS and TEACHER tables. Any student can attend only one class in an academic year and only one teacher takes a class. But a class can have more than 50 students. Now we have to retrieve STUDENT_NAME, ADDRESS, AGE, CLASS_NAME and TEACHER_NAME of each student in a school.

∏STD_NAME, ADDRESS, AGE, CLASS_NAME, TEACHER_NAME ((STUDENT ∞ CLASS_ID CLASS)∞ TECH_IDTEACHER)

Not So efficient

∏STD_NAME, ADDRESS, AGE, CLASS_NAME, TEACHER_NAME (STUDENT ∞ CLASS_ID (CLASS∞ TECH_IDTEACHER))

Efficient

In the first query, it tries to select the records of students from each class. This will result in a very huge intermediary table. This table is then joined with another small table. Hence the traversing of number of records is also more. But in the second query, CLASS and TEACHER are joined first, which has one to one relation here. Hence the number of resulting record is STUDENT table give the final result. Hence this second method is more efficient.

- Sometimes we can combine above heuristic steps with cost based optimization technique to get better results.

All these methods need not be always true. It also depends on the table size, column size, type of selection, projection, join sort, constraints, indexes, statistics etc. Above optimization describes the best way of optimizing the queries.

Measure of Query Cost

- Cost is generally measured as total elapsed time for answering query
    - Many factors contribute to time cost
        - *disk accesses, CPU*, or even network *communication*
- Typically disk access is the predominant cost, and is also relatively easy to estimate. Measured by taking into account
    - Number of seeks * average-seek-cost
    - Number of blocks read * average-block-read-cost
    - Number of blocks written * average-block-write-cost
        - Cost to write a block is greater than cost to read a block
            - data is read back after being written to ensure that the write was successful
- For simplicity we just use the **number of block transfers** *from disk and the* **number of seeks** as the cost measures
    - t_T – time to transfer one block
    - t_S – time for one seek
    - Cost for b block transfers plus S seeks
      $b * t_T + S * t_S$
- We ignore CPU costs for simplicity
    - Real systems do take CPU cost into account
- We do not include cost to writing output to disk in our cost formulae

Query Operations

- Selection Operation
    - The lowest level query processing operator for accessing data is the file scan.
    - search and retrieve records for a given selection condition.
- Linear Search

  Scan each file block and test all records to see whether they satisfy the selection condition.

Cost estimate = b_r block transfers + 1 seek

- b_r denotes number of blocks containing records from relation *r*

If selection is on a key attribute, can stop on finding record

- cost = (b_r/2) block transfers + 1 seek

Linear search can be applied regardless of

- selection condition or
- ordering of records in the file, or
- availability of indices

Selections Using Indices

- **Index scan** – search algorithms that use an index
  - selection condition must be on search-key of index.
- **A2** (**primary index, equality on key**). Retrieve a single record that satisfies the corresponding equality condition
  - *Cost* = (h_i + 1) * (t_T + t_S)
- **A3** (**primary index, equality on nonkey**) Retrieve multiple records.
  - Records will be on consecutive blocks
    - Let b = number of blocks containing matching records
  - *Cost* = h_i * (t_T + t_S) + t_S + t_T * b
- **A4** (**secondary index, equality on nonkey**).
  - Retrieve a single record if the search-key is a candidate key
    - *Cost = (h_i* + 1) * (t_T + t_S)
  - Retrieve multiple records if search-key is not a candidate key
    - each of *n* matching records may be on a different block
    - Cost = (h_i + *n)* * (t_T + t_S)
      - Can be very expensive!

Sorting

- A query may specify that the output should be sorted.
- The processing of some relational query operations can be implemented more efficiently based on sorted relations (join operation)
- For relations that fit into memory, techniques like quick sort can be used.
- For relations that do not fit into memory an external merge sort can be used.

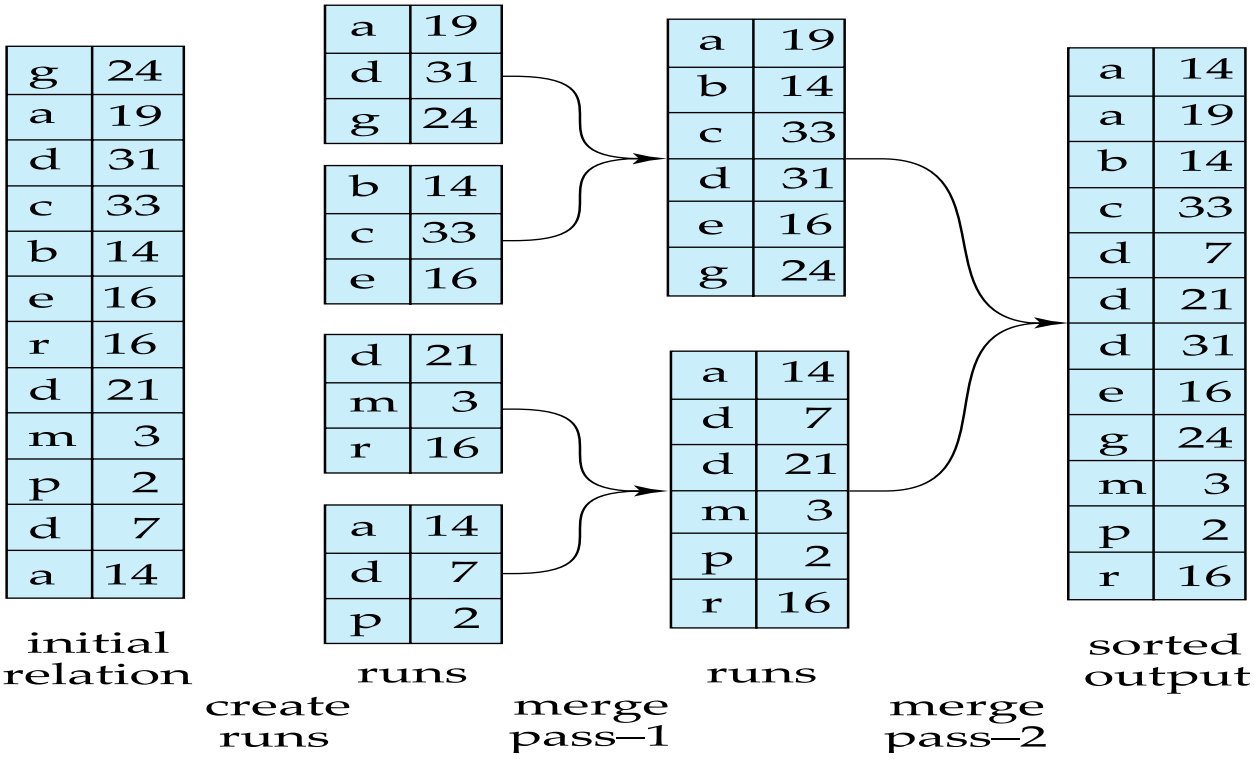
External Merge Sort

Let *M* denote memory size (in pages).

- **Create sorted runs**.
  - Let *i* be 0 initially.
    Repeatedly do the following till the end of the relation:
    (a) Read *M* blocks of relation into memory
    (b) Sort the in-memory blocks
    (c) Write sorted data to run R_i; increment *i.*
    Let the final value of *i* be *N*
- **Merge the runs (N-way merge)**. We assume (for now) that $N < M$.
  - Use *N* blocks of memory to buffer input runs, and 1 block to buffer output. Read the first block of each run into its buffer page
  - **repeat**
    - Select the first record (in sort order) among all buffer pages
    - Write the record to the output buffer. If the output buffer is full write it to disk.
    - Delete the record from its input buffer page.
      If the buffer page becomes empty **then**
      read the next block (if any) of the run into the buffer.
  - **until** all input buffer pages are empty:
- If $N \geq M$, several merge *passes* are required.
  - In each pass, contiguous groups of $M - 1$ runs are merged.
  - A pass reduces the number of runs by a factor of $M - 1$, and creates runs longer by the same factor.

- E.g. If M=11, and there are 90 runs, one pass reduces the number of runs to 9, each 10 times the size of the initial runs

– Repeated passes are performed till all runs have been merged into one.



Query Optimization

- Is an important aspect of query processing.
- Is the activity of choosing an efficient execution strategy for processing a query.
- Aim is to choose one of many equivalent transformations that minimizes resource usage.
- Reduce the total execution time of the query, which is the sum of execution times of all individual operations that make up the query.
- There can be enormous differences in term of performance between different evaluation plans for same query.
    - E.g. seconds vs days to execute same query.
- Cost based query optimization

- Generate logically equivalent expressions by using a set of equivalence rules.
- Annotate the expressions to get alternative query evaluation plans.
- Select the cheapest plan based on the estimated cost.

- Estimation of query evaluation cost based on statistical information from the catalogue manager in combination with the expected performance of algorithms.

Equivalence Rule

- Conjunctive selection operations can be deconstructed into a sequence of individual selections
  - $\sigma_{\theta_1 \wedge \theta_2}(E) = \sigma_{\theta_1}(\sigma_{\theta_2}(E))$
- Selection operations are *commutative*
  - $\sigma_{\theta_1}(\sigma_{\theta_2}(E)) = \sigma_{\theta_2}(\sigma_{\theta_1}(E))$
- Cascade of projection operations (only final one)
  - $\pi_{A_1}(\pi_{A_2}(...(\pi_{A_n}(E))...)) = \pi_{A_1}(E)$
- Selections can be combined with cartesian products and theta joins
  - $\sigma_{\theta}(E_1 \times E_2) = E_1 \bowtie_{\theta} E_2$
  - $\sigma_{\theta_1}(E_1 \bowtie_{\theta_2} E_2) = E_1 \bowtie_{\theta_1 \wedge \theta_2} E_2$
- Theta join (and natural join) operations are *commutative*
  - $E_1 \bowtie_{\theta} E_2 = E_2 \bowtie_{\theta} E_1$
  - note that the order of attributes is ignored
- Natural join operations are *associative*
  - $(E_1 \bowtie E_2) \bowtie E_3 = E_1 \bowtie (E_2 \bowtie E_3)$
- Theta joins are *associative* in the following manner
  - $(E_1 \bowtie_{\theta_1} E_2) \bowtie_{\theta_2 \wedge \theta_3} E_3 = E_1 \bowtie_{\theta_1 \wedge \theta_3} (E_2 \bowtie_{\theta_2} E_3)$
  - where θ_2 contains attributes only from E_2 and E_3
- Union and intersection operations are *commutative*
  - $E_1 \cup E_2 = E_2 \cup E_1$
  - $E_1 \cap E_2 = E_2 \cap E_1$

Union and intersection operations are *associative*

- $(E_1 \cup E_2) \cup E_3 = E_1 \cup (E_2 \cup E_3)$
- $(E_1 \cap E_2) \cap E_3 = E_1 \cap (E_2 \cap E_3)$

The *selection* operation *distributes* over *union*, *intersection* and *set difference*

- $\sigma_P(E_1 - E_2) = \sigma_P(E_1) - \sigma_P(E_2)$

The *projection distributes* over the *union* operation

- $\pi_A(E_1 \cup E_2) = (\pi_A(E_1)) \cup (\pi_A(E_1))$

Note that this is only a selection of equivalence rules

Heuristic Optimization

- Cost-based optimization is expensive, even with dynamic programming.
- Systems may use *heuristics* to reduce the number of choices that must be made in a cost-based fashion.
- Heuristic optimization transforms the query-tree by using a set of rules that typically (but not in all cases) improve execution performance:
  - Perform selection early (reduces the number of tuples)
  - Perform projection early (reduces the number of attributes)
  - Perform most restrictive selection and join operations before other similar operations.
  - Some systems use only heuristics, others combine heuristics with partial cost-based optimization.

Steps in Heuristic Optimization

1. Deconstruct conjunctive selections into a sequence of single selection operations .
2. Move selection operations down the query tree for the earliest possible execution .
3. Execute first those selection and join operations that will produce the smallest relations .
4. Replace Cartesian product operations that are followed by a selection condition by join operations .
5. Deconstruct and move as far down the tree as possible lists of projection attributes, creating new projections where needed .

6. Identify those sub trees whose operations can be pipelined, and execute them using pipelining.

Performance Tuning

- **Database Statistics**
  - Get the correct and updated statistics
- **Create Optimized Indexes**
  - Have a right balance of indexes on tables
- **Specify optimizer hints in SELECT**
  - specify the index name in SELECT query
- **Predetermine expected growth**
  - specify an appropriate value for fill factor when creating indexes
- **Select limited data**
  - Rather than filtering on the client, push as much filtering as possible on the server-end
  - Eliminate any obvious or computed columns
- **Drop indexes before loading data**
  - drop the indexes on a table before loading a large batch of data. This makes the insert statement run faster.
  - Recreate the indexes after insertion.
- **Avoid foreign key constraints**

  Foreign keys constraints ensure data integrity at the cost of performance



Chapter 7(contd..): Query Optimization

Database System Concepts 5th Ed.

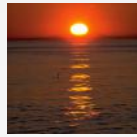
©Silberschatz, Korth and Sudarshan
See www.db-book.com for conditions on re-use





Chapter 14: Query Optimization

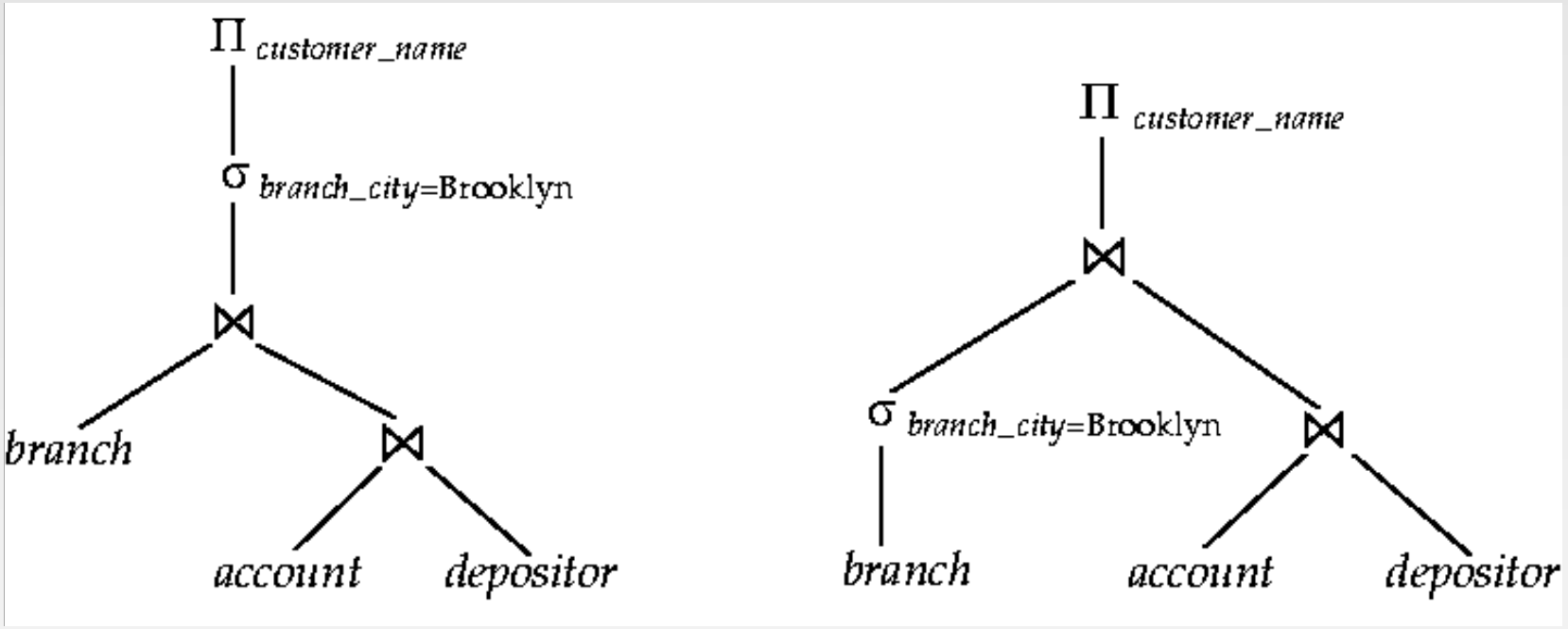
- Introduction
- Transformation of Relational Expressions



©Silberschatz, Korth and Sudarshan

Introduction

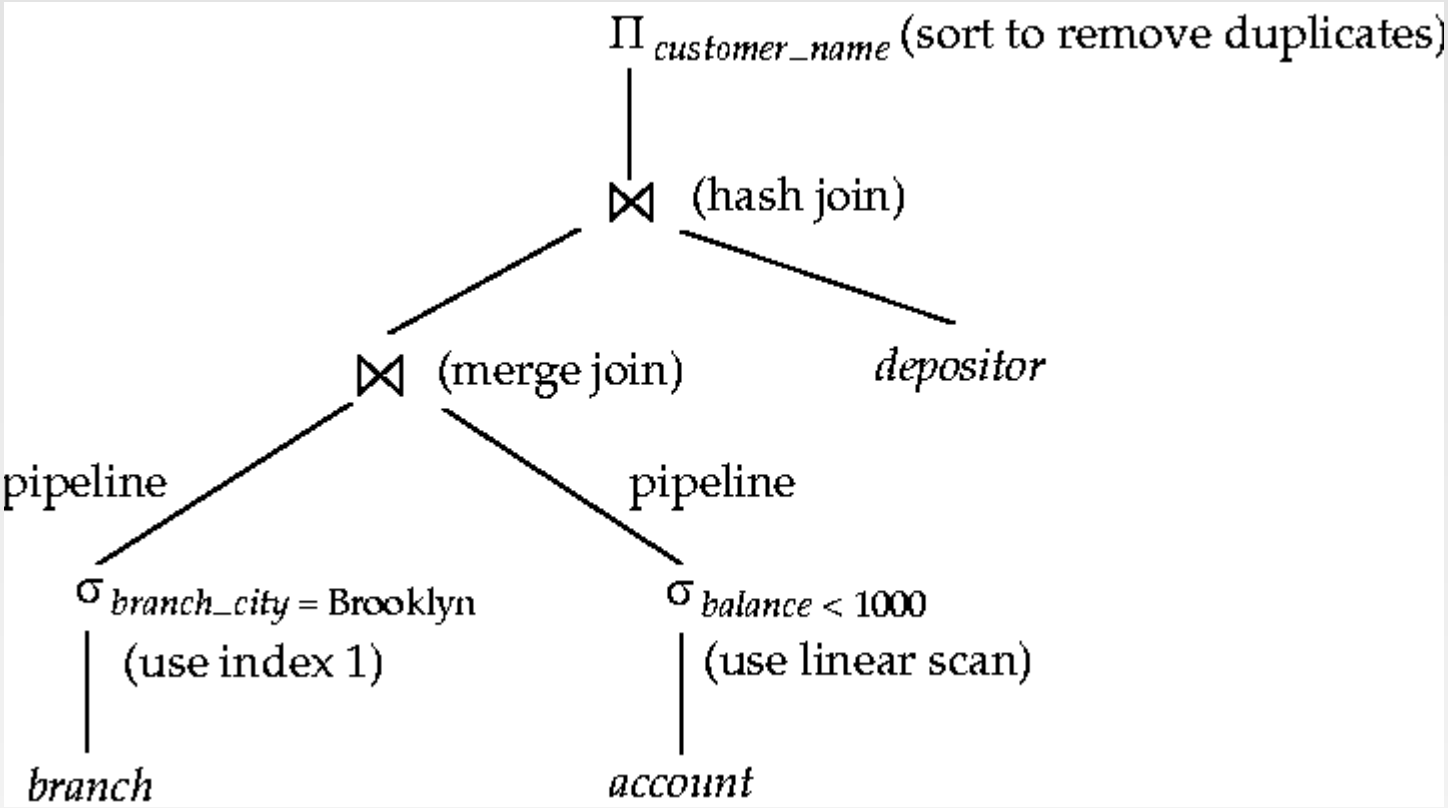
- Alternative ways of evaluating a given query
  - Equivalent expressions
  - Different algorithms for each operation



©Silberschatz, Korth and Sudarshan

Introduction (Cont.)

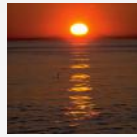
- An **evaluation plan** defines exactly what algorithm is used for each operation, and how the execution of the operations is coordinated.





Introduction (Cont.)

- Cost difference between evaluation plans for a query can be enormous
  - E.g. seconds vs. days in some cases
- Steps in **cost-based query optimization**
  1. Generate logically equivalent expressions using **equivalence rules**
  2. Annotate resultant expressions to get alternative query plans
  3. Choose the cheapest plan based on **estimated cost**
- Estimation of plan cost based on:
  - Statistical information about relations. Examples:
    - number of tuples, number of distinct values for an attribute
  - Statistics estimation for intermediate results
    - to compute cost of complex expressions
  - Cost formulae for algorithms, computed using statistics



©Silberschatz, Korth and Sudarshan



Generating Equivalent Expressions

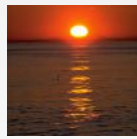
©Silberschatz, Korth and Sudarshan
See www.db-book.com for conditions on re-use





Transformation of Relational Expressions

- Two relational algebra expressions are said to be **equivalent** if the two expressions generate the same set of tuples on every legal database instance
  - Note: order of tuples is irrelevant
- In SQL, inputs and outputs are multisets of tuples
  - Two expressions in the multiset version of the relational algebra are said to be equivalent if the two expressions generate the same multiset of tuples on every legal database instance.
- An **equivalence rule** says that expressions of two forms are equivalent
  - Can replace expression of first form by second, or vice versa



©Silberschatz, Korth and Sudarshan



Equivalence Rules

1. Conjunctive selection operations can be deconstructed into a sequence of individual selections.

$$\sigma_{\theta_1 \wedge \theta_2}(E) = \sigma_{\theta_1}(\sigma_{\theta_2}(E))$$

2. Selection operations are commutative.

$$\sigma_{\theta_1}(\sigma_{\theta_2}(E)) = \sigma_{\theta_2}(\sigma_{\theta_1}(E))$$

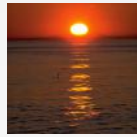
3. Only the last in a sequence of projection operations is needed, the others can be omitted.

$$\Pi_{L_1}(\Pi_{L_2}(\ldots(\Pi_{Ln}(E))\ldots)) = \Pi_{L_1}(E)$$

1. Selections can be combined with Cartesian products and theta joins.

   α. $\sigma_{\theta}(E_1 \times E_2) = E_1 \bowtie_{\theta} E_2$

   β. $\sigma_{\theta 1}(E_1 \bowtie_{\theta 2} E_2) = E_1 \bowtie_{\theta 1 \wedge \theta 2} E_2$



©Silberschatz, Korth and Sudarshan

Equivalence Rules (Cont.)

5. Theta-join operations (and natural joins) are commutative.

$$E_1 \bowtie_\theta E_2 = E_2 \bowtie_\theta E_1$$

6. (a) Natural join operations are associative:

$$(E_1 \bowtie E_2) \bowtie E_3 = E_1 \bowtie (E_2 \bowtie E_3)$$

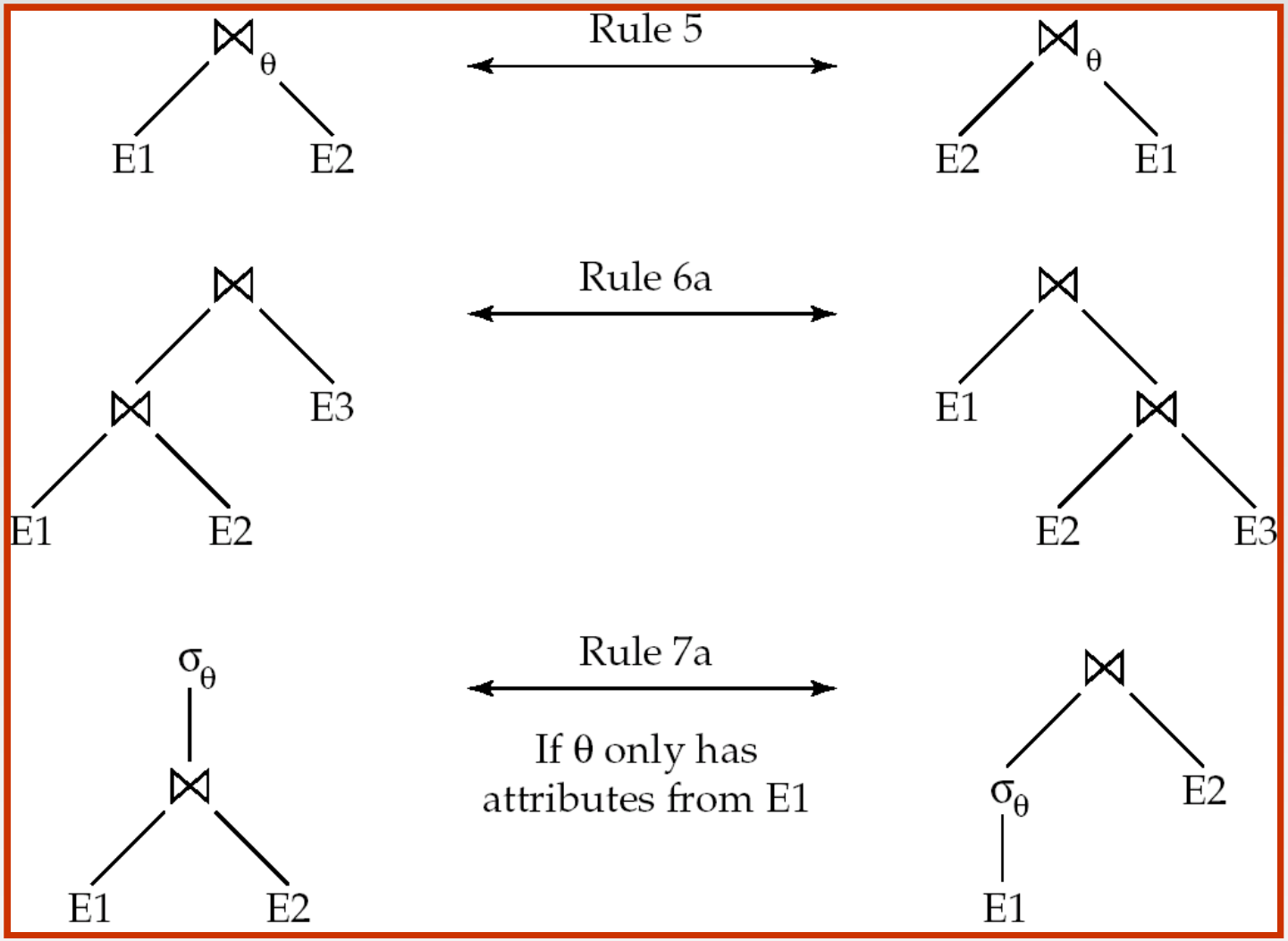
   (b) Theta joins are associative in the following manner:

$$(E_1 \bowtie_{\theta 1} E_2) \bowtie_{\theta 2 \wedge \theta 3} E_3 = E_1 \bowtie_{\theta 1 \wedge \theta 3} (E_2 \bowtie_{\theta 2} E_3)$$

   where θ_2 involves attributes from only E_2 and E_3.

©Silberschatz, Korth and Sudarshan

Pictorial Depiction of Equivalence Rules



©Silberschatz, Korth and Sudarshan

Equivalence Rules (Cont.)

7. The selection operation distributes over the theta join operation under the following two conditions:
   (a) When all the attributes in θ_0 involve only the attributes of one of the expressions (E_1) being joined.

$$\sigma_{\theta_0}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_0}(E_1)) \bowtie_{\theta} E_2$$

   (b) When θ_1 involves only the attributes of E_1 and θ_2 involves only the attributes of E_2.

$$\sigma_{\theta_1 \wedge \theta_2}(E_1 \bowtie_{\theta} E_2) = (\sigma_{\theta_1}(E_1)) \bowtie_{\theta} (\sigma_{\theta_2}(E_2))$$

©Silberschatz, Korth and Sudarshan

Equivalence Rules (Cont.)

8. The projection operation distributes over the theta join operation as follows:

   (a) if θ involves only attributes from $L_1 \cup L_2$:

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_\theta E_2) = (\Pi_{L_1}(E_1)) \bowtie_\theta (\Pi_{L_2}(E_2))$$

   (b) Consider a join $E_1 \bowtie_\theta E_2$.

   - Let L_1 and L_2 be sets of attributes from E_1 and E_2, respectively.
   - Let L_3 be attributes of E_1 that are involved in join condition θ , but are not in $L_1 \cup L_2$, and
   - let L_4 be attributes of E_2 that are involved in join condition θ , but are not in $L_1 \cup L_2$.

$$\Pi_{L_1 \cup L_2}(E_1 \bowtie_\theta E_2) = \Pi_{L_1 \cup L_2}((\Pi_{L_1 \cup L_3}(E_1)) \bowtie_\theta (\Pi_{L_2 \cup L_4}(E_2)))$$

©Silberschatz, Korth and Sudarshan

Equivalence Rules (Cont.)

1. The set operations union and intersection are commutative

$$E_1 \cup E_2 = E_2 \cup E_1$$
$$E_1 \cap E_2 = E_2 \cap E_1$$

   - (set difference is not commutative).

2. Set union and intersection are associative.

$$(E_1 \cup E_2) \cup E_3 = E_1 \cup (E_2 \cup E_3)$$
$$(E_1 \cap E_2) \cap E_3 = E_1 \cap (E_2 \cap E_3)$$

1. The selection operation distributes over $\cup$, $\cap$ and $-$.

$$\sigma_\theta (E_1 - E_2) = \sigma_\theta (E_1) - \sigma_\theta (E_2)$$

   and similarly for $\cup$ and $\cap$ in place of $-$

   Also: $\sigma_\theta (E_1 - E_2) = \sigma_\theta (E_1) - E_2$

   and similarly for $\cap$ in place of $-$, but not for $\cup$

12. The projection operation distributes over union

$$\Pi_L (E_1 \cup E_2) = (\Pi_L (E_1)) \cup (\Pi_L (E_2))$$

©Silberschatz, Korth and Sudarshan

Transformation Example: Pushing Selections

- Query: Find the names of all customers who have an account at some branch located in Brooklyn.

$$\Pi_{customer_name}(\sigma_{branch_city = \text{“Brooklyn”}}(branch \bowtie (account \bowtie depositor)))$$

- Transformation using rule 7a.

$$\Pi_{customer_name}((\sigma_{branch_city = \text{“Brooklyn”}}(branch)) \bowtie (account \bowtie depositor))$$

- Performing the selection as early as possible reduces the size of the relation to be joined.

©Silberschatz, Korth and Sudarshan

Example with Multiple Transformations

- Query: Find the names of all customers with an account at a Brooklyn branch whose account balance is over $1000.

  $\Pi_{customer_name}(\sigma_{branch_city = \text{“Brooklyn”} \wedge balance > 1000}(branch \bowtie (account \bowtie depositor)))$

- Transformation using join associatively (Rule 6a):

  $\Pi_{customer_name}((\sigma_{branch_city = \text{“Brooklyn”} \wedge balance > 1000}(branch \bowtie account)) \bowtie depositor)$

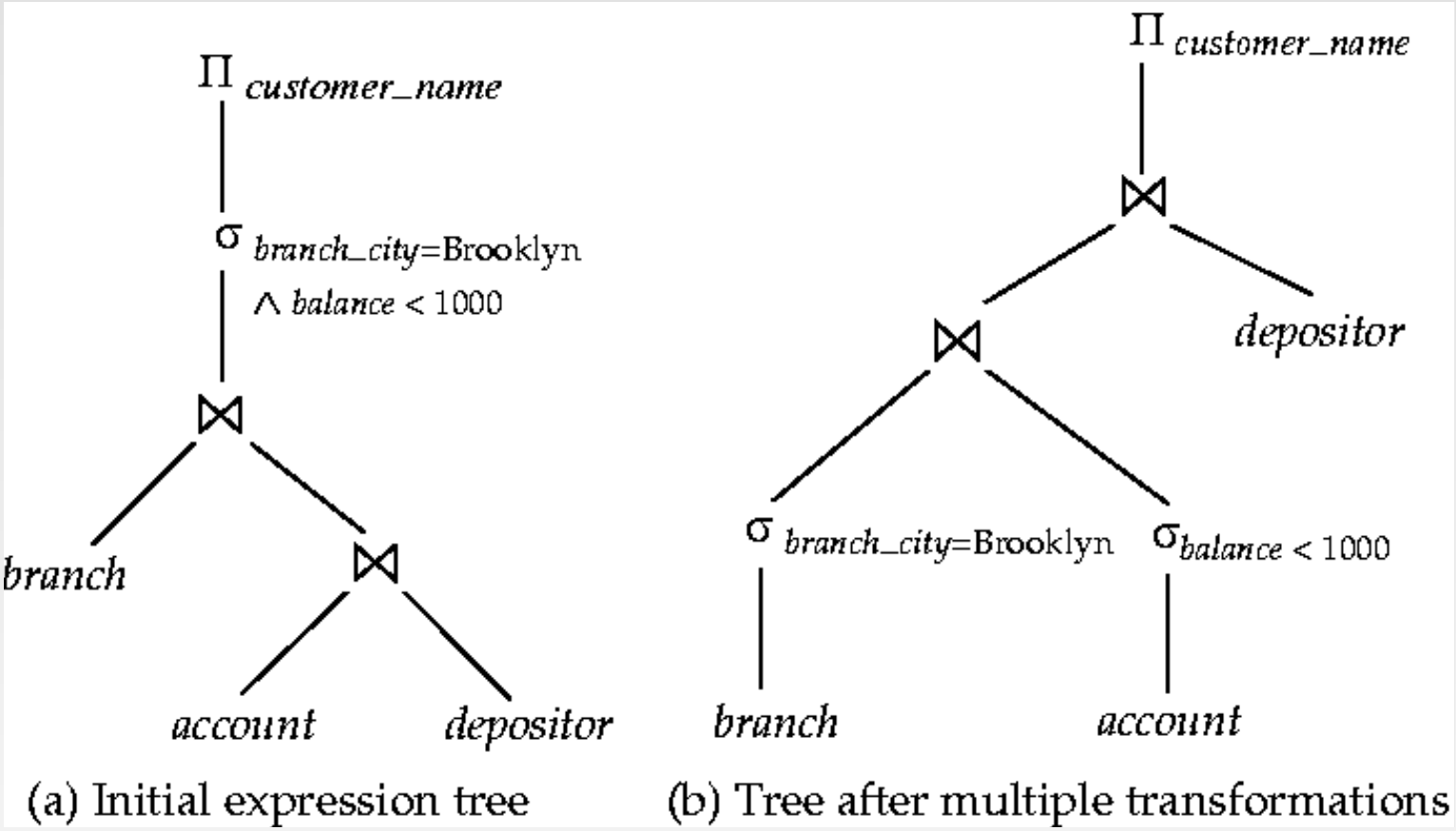
- Second form provides an opportunity to apply the “perform selections early” rule, resulting in the subexpression

  $\sigma_{branch_city = \text{“Brooklyn”}}(branch) \bowtie \sigma_{balance > 1000}(account)$

- Thus a sequence of transformations can be useful

©Silberschatz, Korth and Sudarshan

Multiple Transformations (Cont.)



©Silberschatz, Korth and Sudarshan

Transformation Example: Pushing Projections

$$\Pi_{customer_name}((\sigma_{branch_city = \text{“Brooklyn”}}(branch)) \bowtie account) \bowtie depositor)$$

- When we compute

  $(\sigma_{branch_city = \text{“Brooklyn”}}(branch)) \bowtie account)$

  we obtain a relation whose schema is:
  (*branch_name, branch_city, assets, account_number, balance)*
- Push projections using equivalence rules 8a and 8b; eliminate unneeded attributes from intermediate results to get:

  $\Pi_{customer_name}(($
  $\Pi_{account_number}((\sigma_{branch_city = \text{“Brooklyn”}}(branch)) \bowtie account)) \bowtie depositor)$
- Performing the projection as early as possible reduces the size of the relation to be joined.

©Silberschatz, Korth and Sudarshan



End of Chapter

©Silberschatz, Korth and Sudarshan
See www.db-book.com for conditions on re-use

